

Lab 4 Report

Lab 4

12532588 李子豪

3. 实现结果

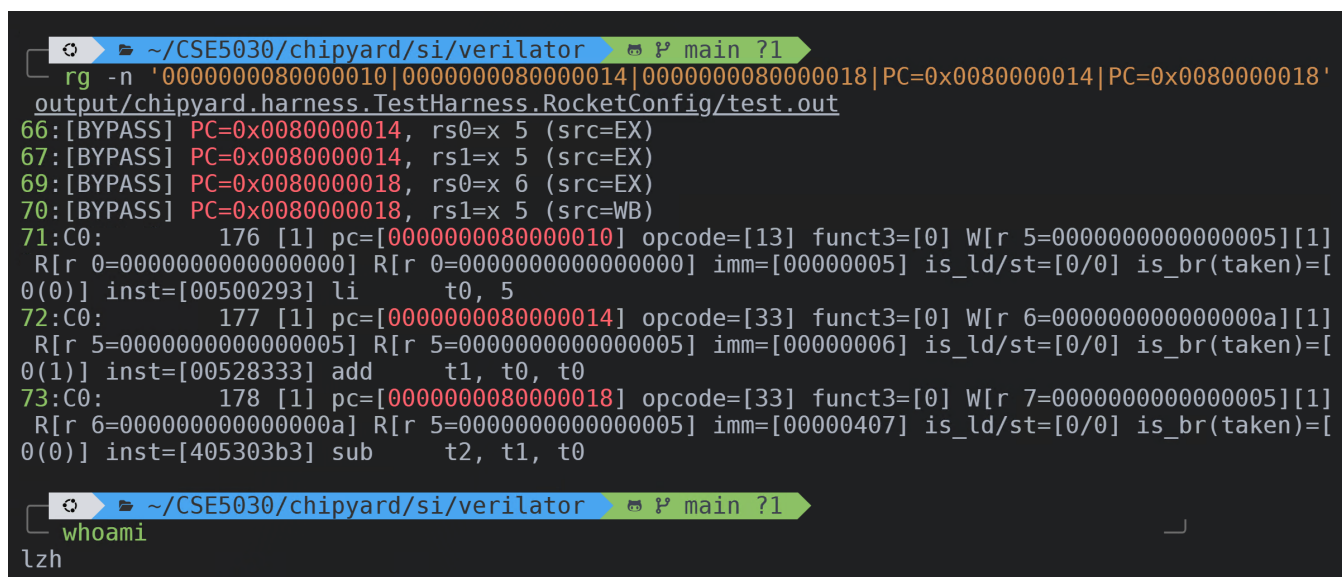
3.1 Data Hazard

汇编代码

```
14      # 1. Data hazard:
15      # addi writes t0, the next add uses t0 immediately.
16      addi t0, zero, 5
17      add t1, t0, t0
18      sub t2, t1, t0
19
```

该代码构造了普通 ALU 数据相关：addi 先写入 t0，后续 add 与 sub 立即读取相关寄存器，用于观察 bypass 行为。

截图



```
rg -n '0000000080000010|0000000080000014|0000000080000018|PC=0x0080000014|PC=0x0080000018'
output/chipyard.harness.TestHarness.RocketConfig/test.out
66:[BYPASS] PC=0x0080000014, rs0=x 5 (src=EX)
67:[BYPASS] PC=0x0080000014, rs1=x 5 (src=EX)
69:[BYPASS] PC=0x0080000018, rs0=x 6 (src=EX)
70:[BYPASS] PC=0x0080000018, rs1=x 5 (src=WB)
71:C0:      176 [1] pc=[0000000080000010] opcode=[13] funct3=[0] W[r 5=0000000000000005][1]
R[r 0=0000000000000000] R[r 0=0000000000000000] imm=[00000005] is_ld/st=[0/0] is_br(taken)=[
0(0)] inst=[00500293] li      t0, 5
72:C0:      177 [1] pc=[0000000080000014] opcode=[33] funct3=[0] W[r 6=000000000000000a][1]
R[r 5=0000000000000005] R[r 5=0000000000000005] imm=[00000006] is_ld/st=[0/0] is_br(taken)=[
0(1)] inst=[00528333] add     t1, t0, t0
73:C0:      178 [1] pc=[0000000080000018] opcode=[33] funct3=[0] W[r 7=0000000000000005][1]
R[r 6=000000000000000a] R[r 5=0000000000000005] imm=[00000407] is_ld/st=[0/0] is_br(taken)=[
0(0)] inst=[405303b3] sub     t2, t1, t0

whoami
lzh
```

日志中，PC=0x0080000014 出现 src=EX，PC=0x0080000018 同时出现 src=EX 和 src=WB。对应的 WB 周期为 176 -> 177 -> 178，说明发生了数据旁路，但没有插入 stall。另外，BYPASS 日志反映的是 EX 阶段的旁路选择，而指令日志反映的是 WB 阶段的提交结果，因此两类日志在时间上会自然错开。

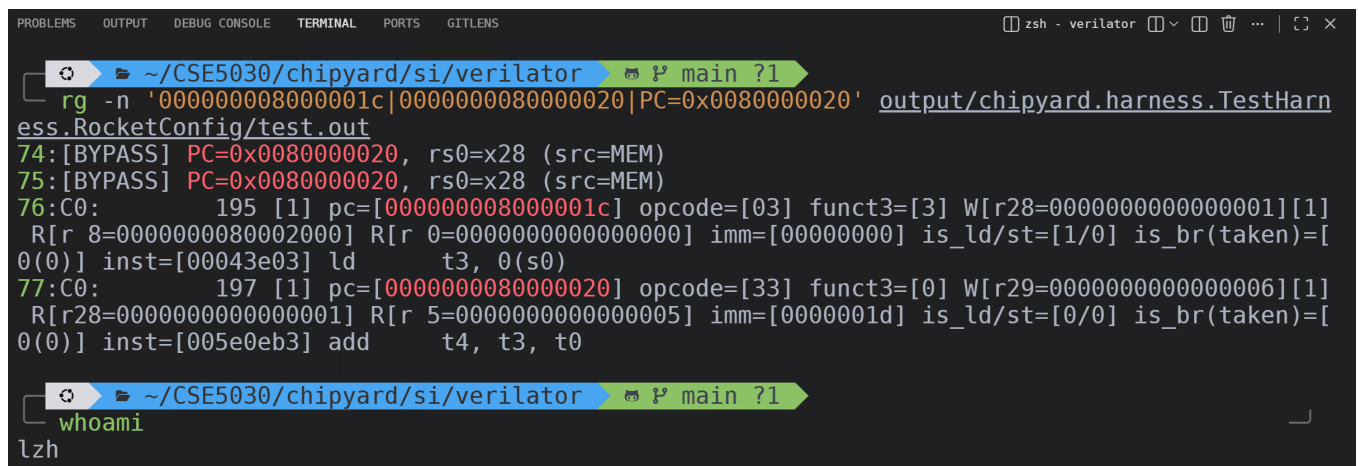
3.2 Load-Use Data Hazard

汇编代码

```
20      # 2. Load-use hazard:
21      # ld writes t3, the next add uses t3 immediately.
22      ld t3, 0(s0)
23      add t4, t3, t0
```

该代码构造了典型的 load-use 相关：`ld t3, 0(s0)` 从内存读取数据后，下一条 `add t4, t3, t0` 立即使用 `t3`，用于观察是否引入停顿。

截图



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS zsh - verilator
rg -n '000000008000001c|0000000080000020|PC=0x0080000020' output/chipyard.harness.TestHarn
ess.RocketConfig/test.out
74:[BYPASS] PC=0x0080000020, rs0=x28 (src=MEM)
75:[BYPASS] PC=0x0080000020, rs0=x28 (src=MEM)
76:C0:      195 [1] pc=[000000008000001c] opcode=[03] funct3=[3] W[r28=0000000000000001][1]
R[r 8=0000000080002000] R[r 0=0000000000000000] imm=[00000000] is_ld/st=[1/0] is_br(taken)=[
0(0)] inst=[00043e03] ld      t3, 0(s0)
77:C0:      197 [1] pc=[0000000080000020] opcode=[33] funct3=[0] W[r29=0000000000000006][1]
R[r28=0000000000000001] R[r 5=0000000000000005] imm=[0000001d] is_ld/st=[0/0] is_br(taken)=[
0(0)] inst=[005e0eb3] add     t4, t3, t0

whoami
lzh
```

日志中，`PC=0x0080000020` 的 bypass 来源是 `MEM`，同时 `ld` 与后续 `add` 的 WB 周期分别为 195 和 197。这说明 load 的结果要到 MEM 阶段才可用，而后一条 `add` 会在更早的 EX 阶段就需要 `t3`，因此产生了 stall。

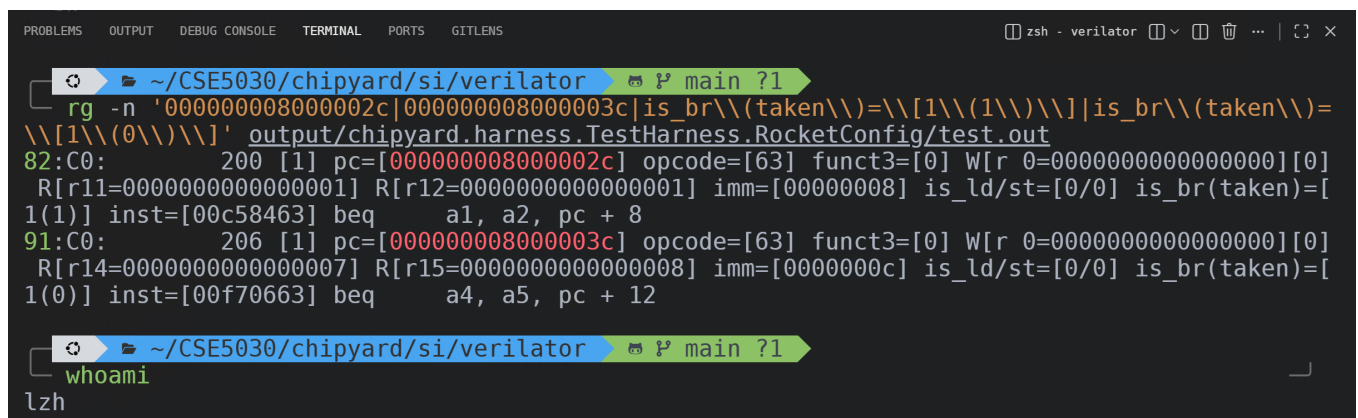
3.3 Control Hazard

汇编代码

```
25      # 3. Control hazard:
26      # First branch is taken, second branch is not taken.
27      addi a1, zero, 1
28      addi a2, zero, 1
29      beq a1, a2, taken_branch
30      addi a3, zero, 99
31
32  taken_branch:
33      addi a4, zero, 7
34      addi a5, zero, 8
35      beq a4, a5, not_taken_target
36      addi a6, zero, 1
37      j branch_done
38
39  not_taken_target:
40      addi a6, zero, 2
41
42  branch_done:
43      sw zero, 0(s1)
```

该代码构造了两条前向分支：第一条 `beq a1, a2, taken_branch` 实际 taken，第二条 `beq a4, a5, not_taken_target` 实际 not taken，用于对照观察 control hazard 和默认预测路径。

截图



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
zsh - verilator

[?] main ?1
rg -n '000000008000002c|000000008000003c|is_br\\(taken\\)=\\[1\\(1\\)\\]|is_br\\(taken\\)=\\[1\\(0\\)\\]' output/chipyard.harness.TestHarness.RocketConfig/test.out
82:C0: 200 [1] pc=[000000008000002c] opcode=[63] funct3=[0] W[r 0=0000000000000000][0] R[r11=00000000000000001] R[r12=00000000000000001] imm=[000000008] is_ld/st=[0/0] is_br(taken)=[1(1)] inst=[00c58463] beq a1, a2, pc + 8
91:C0: 206 [1] pc=[000000008000003c] opcode=[63] funct3=[0] W[r 0=0000000000000000][0] R[r14=00000000000000007] R[r15=00000000000000008] imm=[00000000c] is_ld/st=[0/0] is_br(taken)=[1(0)] inst=[00f70663] beq a4, a5, pc + 12

[?] main ?1
whoami
lzh
```

日志中，`pc=[000000008000002c]` 的 branch 实际 taken，之后进入目标地址 `pc=[0000000080000034]`；`pc=[000000008000003c]` 的 branch 实际 not taken，之后继续执行顺序地址 `pc=[0000000080000040]`。这可用于分析 Rocket 的默认分支预测策略。

4. Hazard 分析

4.1 Data Hazards

该 **hazard** 是否导致了 **pipeline stall** ?

没有导致明显的 pipeline stall。

为什么会或不会发生 **stall** ?

因为这组相关属于普通 ALU 指令之间的数据相关，后一条指令需要的结果可以通过 bypass 直接获得，不需要像 load-use hazard 那样等待数据从内存阶段返回。从 WB 日志也可以看到三条相关指令的周期分别是 C0: 176、C0: 177、C0: 178，周期连续，没有插入额外气泡。

bypass 来自哪个 **pipeline stage** ?

从日志可见，bypass 主要来自 EX，在后续依赖中还出现了 WB。

结合指令序列，解释为什么 **bypass** 来自这个 **stage**

addi t0, zero, 5 在前一条指令中生成 t0，紧随其后的 add t1, t0, t0 立即读取 t0，因此日志中在 PC=0x0080000014 出现 src=EX。随后 sub t2, t1, t0 同时依赖 t1 和 t0，因此在 PC=0x0080000018 可以看到 EX 和 WB 两种 bypass 来源。

4.2 Load-Use Data Hazards

该 **hazard** 是否导致了 **pipeline stall** ?

导致了 stall。

为什么会或不会发生 **stall** ?

因为 ld t3, 0(s0) 的数据不会在 EX 阶段立即产生，而是要到 MEM 阶段才可用；但紧接着的 add t4, t3, t0 会在自己的 EX 阶段就读取 t3。因此消费者取数时，load 结果还没准备好，必须插入一个 stall cycle。

日志中的哪些证据体现了 **stall cycles** ?

从日志可见，ld 位于 pc=[000000008000001c]，其对应的 WB 周期是 C0: 195；紧随其后的 add 位于 pc=[0000000080000020]，其对应周期是 C0: 197。与前面的普通 ALU 相关相比，这里出现了更明显的间隔。同时，bypass 来源为 MEM，也说明 load 的结果直到 MEM 阶段才可被后续指令转发。

4.3 Control Hazards

默认的 **branch prediction policy** 是什么？

从实验日志推断，默认的静态 branch prediction policy 是 `predict not taken`。

你是如何从日志中判断出来的？

从两条前向分支的执行结果可以推断这一点。第一条 `beq a1, a2, pc + 8` 实际 taken，日志显示该分支位于 `pc=[000000008000002c]`，其后进入目标地址 `pc=[0000000080000034]`；这说明顺序路径被纠正，符合“先按 not taken 方向取指，再在分支判定后跳转”的现象。第二条 `beq a4, a5, pc + 12` 实际 not taken，日志中在 `pc=[000000008000003c]` 之后继续执行顺序地址 `pc=[0000000080000040]`，与 `predict not taken` 完全一致。因此可以根据 log 判断默认策略是 `predict not taken`。

taken branch 分析

在 `pc=[000000008000002c]` 的 `beq a1, a2, pc + 8` 中，`a1` 和 `a2` 都被设置为 1，因此分支实际发生跳转。日志中的 `is_br(taken)=[1(1)]` 表示这是一条控制流指令，并且这一次 branch 实际被 taken。之后出现的是目标地址上的 `pc=[0000000080000034]`，说明分支最终跳到了目标块。

not taken branch 分析

在 `pc=[000000008000003c]` 的 `beq a4, a5, pc + 12` 中，`a4=7`、`a5=8`，因此分支实际不跳转。日志中的 `is_br(taken)=[1(0)]` 表示这是一条控制流指令，但这一次 branch 实际没有 taken。之后继续执行顺序地址上的 `pc=[0000000080000040]`，因此这条分支与默认的 `predict not taken` 路径一致，也是判断默认静态预测策略的关键证据。